Operating Systems Interview Guide

Concepts explained simply, key comparisons,
solved numericals and the top 50 interview questions

PROCESSES | THREADS | SCHEDULING | MEMORY | DEADLOCKS

137 core questions and answers

Comparison tables and cheat sheets

50 top interview questions + output-prediction puzzles

Solved scheduling and Banker's numericals

Contents

Part 1: Operating System Basics 3

Part 2: Process Fundamentals 4

Part 3: Threads and Thread Internals 7

Part 4: CPU Scheduling 9

Part 5: Process Synchronization and IPC 12

Part 6: Deadlocks and Livelock 16

Part 7: Memory Management and Virtual Memory 17

Part 8: System Calls, Kernel, Boot and Hardware 19

Part 9: File Systems, Linking and Background Processes 20

Part 10: Quick Revision Cheat Sheets 21

Part 11: Top 50 Operating Systems Interview Questions 23

How to use this guide

Questions are numbered Q1 to Q137 in the same order as your notes, grouped into nine parts. Examples are kept only where they genuinely help (process vs thread, fork, race condition, convoy effect, deadlock). Part 10 has quick-revision cheat sheets and Part 11 closes with the top 50 most-asked OS interview questions, followed by solved numericals (scheduling, Banker's algorithm) and output-prediction questions.

Part 1: Operating System Basics

Q1. What is an Operating System?

An Operating System is a software program that manages hardware and software resources. It acts as an intermediary between the user and hardware systems. It handles basic tasks such as memory management, process management, device management, and input/output operations.

Q2. Can I switch from one operating system to another?

Yes, it is possible to switch from one operating system to another. However, it may require reinstalling the new operating system and ensuring compatibility with your hardware and software requirements.

Q3. Can I install multiple operating systems on a single computer?

Yes, it is possible to install multiple operating systems on a single computer using techniques such as dual booting or virtualization.

Q4. What are the popular Operating Systems?

Windows, Linux, macOS, Android, iOS.

Q5. Why do we need Operating Systems?

They are required to manage resources efficiently, provide user-friendly interfaces, and ensure security.

Q6. What are the different types of Operating Systems?

Batch System, Distributed System, Time-Sharing System, Desktop System, Real-Time Operating System.

Q7. What is a Batch System?

A batch system is an operating system that processes a set of similar tasks or jobs in batch mode without any user interaction.

Q8. What is a Real-Time Operating System?

It prioritizes predictability and timely execution for critical systems and is specifically designed to handle tasks with strict timing requirements.

Q9. What is a Distributed System?

A distributed operating system runs on multiple interconnected computers and enables them to work together as a single cohesive unit.

Q10. What is a Time-Sharing System?

A time-sharing system, also known as a multitasking system, allows multiple users to concurrently share the resources of a single computer.

Q11. What is a Desktop System?

A desktop system refers to an operating system designed for personal computers or workstations used by individual users.

Part 2: Process Fundamentals

Q12. What is a Process?

A process is a program that is running on your computer (e.g., Notepad, a browser). It allows you to do things like open applications, play music, and browse the web - all at the same time. A process is created when you open a program and removed when you close it.

Q13. What are the different states of a Process?

- **New:** Initial state where the process is created.
- **Ready:** Loaded into memory and waiting to be assigned to a processor.
- **Running:** Being executed by the processor.
- **Blocked:** Unable to execute further until a certain event occurs.
- **Terminated:** Has finished execution and is removed from the system.

Transition	Trigger
New -> Ready	Admitted by the long-term scheduler
Ready -> Running	Dispatched by the short-term scheduler
Running -> Ready	Time slice expires or a higher-priority process arrives (preemption)
Running -> Blocked	Waits for I/O or an event
Blocked -> Ready	I/O or event completes
Running -> Terminated	Process finishes or is killed

Q14. What are the elements of a Process?

- **Heap:** Dynamically allocated memory/object to the process.
- **Stack:** Stores temporary data such as local variables, CPU register, Program Counter, and function calls.
- **Data:** Stores static and global variables.
- **Text:** Contains the program code and instructions (program counter points here).

Section	Holds	Grows/Notes
Stack	Local variables, function calls, return addresses	Grows downward, LIFO
Heap	Dynamically allocated memory (malloc/new)	Grows upward
Data	Global and static variables	Fixed size
Text	Program code (instructions)	Read-only

Q15. How does a Process differ from a Program?

A program is just a set of instructions saved in a file. A process is what that program becomes when it starts running. So, a process is an active instance of a program.

Program	Process
Passive (a file on disk)	Active (running in memory)
Exists until deleted	Exists only while executing
Needs only storage	Needs CPU, memory, I/O
One program can create many processes	Each process has its own PCB

Q16. What are the stages of a Process?

1. The process is submitted to the process queue.
2. Once submission is acknowledged, the "new" state is assigned.
3. It then moves to the "ready" state.
4. A processor is assigned to the process in the ready state.
5. Execution of the process by the processor is called the "running" state.
6. **Wait state:** The process may enter a wait state to access input, when interrupted by the OS, or when a resource/memory is locked by another process.
7. **Terminated state:** After the process has finished executing.

Q17. Can a Process move from Running back to Ready?

Yes, if the CPU needs to switch to another process (as in multitasking).

Q18. Why is the Process lifecycle important?

It helps the operating system manage processes efficiently and handle many tasks at once.

Q19. What is a Process Control Block (PCB)?

A Process Control Block is a data structure used by operating systems to track a process. It contains the Process ID (PID), program counter, memory allocation, process state, I/O status, registers, and more. It acts like an identity card for the process.

Q20. Why is the PCB important in OS?

It allows the Operating System to keep track of a process's state so it isn't lost when the OS switches to other processes.

Q21. Can a user see or change the PCB?

No, users cannot directly access or view the PCB. It is managed by the kernel.

Q22. What is a Zombie Process?

A zombie process is one that has completed execution but still has an entry in the process table so its exit status can be reported to its parent. Every child process briefly becomes a zombie until the parent process reads its exit status and removes the entry.

> The parent "reaps" the zombie by calling `wait()` / `waitpid()`. A zombie uses no CPU or memory, only a process-table slot, but too many zombies can exhaust PIDs.

Q23. What is an Orphan Process?

An orphan process is a process whose parent has terminated or exited before the child process completed execution. **Handling Orphan Processes:** Operating systems handle orphan processes automatically through a process called re-parenting. The system's top-level process (init/systemd on Linux) immediately adopts the orphaned child and cleans up its entry from the process table when it completes.

Q24. What is the difference between Multiprogramming, Multitasking, and Multiprocessing?

- **Multiprogramming (maximizing CPU usage):** Keeps multiple programs loaded in main memory (RAM) at the same time. The CPU executes one program until that program needs to wait for an external operation (like reading from a hard drive or waiting for user input). Instead of letting the CPU sit idle, the OS context-switches to another program stored in memory. *Analogy:* a receptionist who answers the second phone line while the first caller is on hold.
- **Multitasking / Time-sharing (concurrent execution):** An extension of multiprogramming designed for user interactivity. The OS divides CPU time into tiny intervals called time slices or quanta (a few milliseconds). The CPU switches between active tasks so rapidly that users feel all applications (browser, music player, word

processor) are running simultaneously on a single processor. *Analogy:* a chef preparing three dishes on one burner, switching the pan every 30 seconds.

- **Multiprocessing (parallel execution):** Using two or more physical CPUs (or multi-core processors) within a single computer system. True hardware parallelism: separate processes or threads run on physically separate cores at the exact same instant. *Analogy:* three chefs at three burners cooking at the same time.

Aspect	Multiprogramming	Multitasking	Multiprocessing
CPUs	1	1	2 or more
Switch trigger	Process waits for I/O	Time slice expires	Not needed (true parallel)
Goal	Max CPU utilization	Responsiveness	Throughput and speed

Q25. Explain fork() in OS?

`fork()` is a system call that creates a new process (the "child") by duplicating the calling process (the "parent"). It returns **0** in the child, the **child's PID** in the parent, and **-1** if it fails.

```
pid_t pid = fork();
if (pid == 0)       printf("I am the child\n");
else if (pid > 0)   printf("I am the parent, child PID = %d\n", pid);
else                perror("fork failed");
```

Q26. What is Copy-on-Write, and where is it used?

Copy-on-Write (COW) is an optimization technique where multiple processes initially share the same memory pages; a page is only duplicated when one of the processes attempts to modify it. This is commonly used in the `fork()` system call to avoid unnecessarily copying a parent process's entire memory space when creating a child process.

Part 3: Threads and Thread Internals

Q27. What is the difference between a Process and a Thread?

A process is a complete program with its own memory. A thread is a smaller unit within a process that shares the same memory. Threads allow a process to perform multiple tasks simultaneously.

> **Example: Web browser**
> Each tab can run as a separate **process** (a crash in one tab does not kill the others). Inside one tab, separate **threads** handle rendering, JavaScript, network requests and the UI, all sharing that tab's memory.

Aspect	Process	Thread
Memory	Own address space	Shares the process's address space (code, data, heap)
Private to it	Everything	Stack, registers, program counter
Creation cost	High	Low
Context switch	Slow (memory maps change)	Fast
Communication	IPC needed (pipes, sockets, shared memory)	Direct via shared memory
Isolation	One crash does not affect others	One thread crash can kill the whole process

Q28. What is a Thread?

A thread is the smallest execution unit of a program that performs a task. It shares the memory of its parent process. It has its own stack, state, program counter, and CPU registers. Types of threads: User-Level Thread and Kernel-Level Thread.

Q29. Why is process creation more expensive than thread creation?

Creating a new process needs extra address space and dedicated system resources in the operating system, whereas threads share the parent process's address space.

Q30. What happens when a thread actually runs?

Java Code -> JVM -> OS thread creation -> Thread becomes schedulable -> Scheduler selects it -> CPU executes it.

Q31. What is a TCB (Thread Control Block)?

A Thread Control Block keeps track of all threads in the system. It contains information such as thread ID, thread state, stack pointer, program counter, CPU registers, and priority.

Q32. Where does the Thread live?

In Java, the JVM holds the Thread object in heap memory, while the corresponding native thread is managed by the operating system kernel.

Q33. Why do we need synchronization if each thread has its own state?

Because stack memory is private to each thread, but heap memory is shared. To ensure shared heap memory is accessed properly and safely, synchronization is required.

Q34. What is the difference between a CPU Thread and an I/O Thread?

- **CPU Threads:** Spend most of their time handling computations, mathematical operations, image processing, or compression.
- **I/O Threads:** Spend most of their time waiting for input/output operations like HTTP requests, database queries, network sockets, or file reads.

Q35. What is a User-Level Thread?

A thread created and managed in user space without direct kernel involvement is called a user-level thread. It offers faster creation and thread switching since it avoids kernel context switches. Example: Early Java implementations used Green Threads managed purely in user space.

Part 4: CPU Scheduling

Q36. What is Process Scheduling?

It is the process of deciding which process should be executed by the processor and for how long. Common algorithms include FCFS, Round Robin, SJF, and Priority Scheduling.

Q37. What is Context Switching?

When a CPU switches from one process to another, it is called context switching. The current process's state is saved so it can resume later, and the next process's state is loaded and started.

Q38. What happens during Context Switching?

The OS saves the current execution state (registers, program counter, PCB) of the running process and loads the saved state of the next process to be executed.

Q39. What are the types of Process Scheduling?

- **Preemptive:** The currently running process can be interrupted and switched out by the OS before completing.
- **Non-Preemptive:** The running process cannot be interrupted; the CPU must wait until it voluntarily yields or completes execution.

Q40. What are the different types of Queues in OS?

- **Job Queue:** Contains all jobs entering the system.
- **Ready Queue:** Contains processes residing in main memory waiting for CPU execution time.
- **Device Queue:** Contains processes waiting for a specific I/O device.

Q41. What are the different types of Schedulers?

- **Long-Term Scheduler (Job Scheduler):** Decides which processes are admitted into main memory from the disk pool.
- **Short-Term Scheduler (CPU Scheduler):** Decides which process in the ready queue gets CPU execution time next.
- **Medium-Term Scheduler (Swapping Scheduler):** Moves blocked or suspended processes from main memory to secondary storage (swapping out) and back (swapping in).

Q42. What happens if there is no proper scheduling?

- Some jobs may get stuck indefinitely (starvation).
- Applications may freeze, lag, or become unresponsive.
- Overall system throughput and CPU utilization degrade significantly.

Q43. What problems can occur in CPU scheduling?

- **Starvation:** Low-priority processes never get CPU execution time.
- **Convoy Effect:** Short processes wait behind a long, CPU-bound process in FCFS.
- **High Overhead:** Frequent context switching (e.g., extremely short quantum in Round Robin) wastes CPU cycles.

Q44. What is First Come First Serve (FCFS)?

This is the simplest scheduling algorithm. As the name suggests, the process that arrives and enters the ready queue first gets executed by the CPU first.

Q45. What is the Convoy Effect?

The Convoy Effect occurs in FCFS when a process with a large burst time arrives first, causing all subsequent shorter processes to wait in line for a long duration.

Example: Convoy effect in numbers
P1 (burst 24 ms), P2 (3 ms), P3 (3 ms) all arrive at time 0. In FCFS order P1, P2, P3 the waiting times are 0, 24, 27 so the average is **17 ms**. If the short jobs ran first (P2, P3, P1) the waits would be 0, 3, 6 so the average drops to **3 ms**.

Q46. What is the main drawback of FCFS?

The Convoy Effect is the primary disadvantage, leading to high average waiting times for shorter processes.

Q47. When is FCFS preferred?

In batch processing systems, embedded control systems, and simple single-tasking environments where predictability and low overhead are prioritized.

Q48. How does FCFS differ from other scheduling algorithms?

FCFS executes processes strictly in arrival order non-preemptively. Round Robin uses time slicing, Shortest Job First prioritizes shorter burst times, and Priority Scheduling selects processes based on assigned priority levels.

Q49. What is the Shortest Remaining Time First (SRTF)?

SRTF is the preemptive version of Shortest Job First (SJF). It picks the process with the smallest remaining CPU burst time. If a new process arrives with a shorter remaining time than the currently running process, the current process is preempted.

Q50. What is Non-Preemptive Shortest Job First (SJF)?

In Non-Preemptive SJF, the scheduler selects the process with the shortest CPU burst time from the ready queue, but once a process starts executing, it runs to completion without interruption.

Q51. Is Preemptive SJF used in modern operating systems?

Preemptive SJF is optimal for minimizing average waiting time in theory, but it is rarely used in general-purpose OSs because predicting exact future CPU burst times is impractical.

Q52. What is the key difference between Preemptive (SRTF) and Non-Preemptive (SJF)?

In Preemptive scheduling (SRTF), a running process can be interrupted if a newly arrived process has a shorter remaining burst time. In Non-Preemptive scheduling (SJF), the executing process keeps the CPU until it finishes or yields voluntarily.

Q53. How can we prevent starvation in the Shortest Job First?

Starvation occurs when long processes wait indefinitely while shorter ones keep arriving. This is solved using Aging, a technique where the priority of a waiting process is gradually increased over time.

Q54. What is the Round Robin Algorithm?

Each process in the ready queue is assigned a fixed time quantum. The CPU executes a process for that duration; if the process does not finish within the quantum, it is preempted and moved to the back of the queue.

Q55. What is the main advantage of Round Robin?

It provides good response time for interactive applications and ensures fairness, as every process gets an equal share of CPU time, preventing starvation.

Q56. What is Priority Scheduling?

Priority Scheduling assigns a priority level to each process. The CPU is allocated to the process with the highest priority (either preemptively or non-preemptively).

Q57. How does Aging solve the Starvation problem?

Aging incrementally increases the priority level of processes that have been waiting in the queue for a long time, guaranteeing that even the lowest priority process eventually gets executed.

Q58. How does the Convoy Effect differ from Starvation?

The Convoy Effect causes delays across multiple short processes because they are blocked behind one large process. Starvation refers to a specific process being indefinitely denied CPU time because higher-priority tasks keep preempting it.

Q59. Why is FCFS preferred despite its major drawbacks?

FCFS is extremely simple to implement, has zero algorithm overhead, and performs predictably well when processes have similar execution times.

Q60. What is a Scheduler?

A scheduler is an OS module responsible for selecting which process from the ready queue should be allocated to the CPU for execution.

Q61. What is a Dispatcher?

A dispatcher is the component that takes over control after the scheduler selects a process. It handles context switching, switching to user mode, and jumping to the proper location in the program to start execution.

> Scheduler = decides WHO runs next. Dispatcher = actually GIVES the CPU to that process. The time the dispatcher takes is called dispatch latency.

Q62. What happens if the dispatcher fails during context switching?

The operating system can crash, freeze, or experience a kernel panic because the system state becomes corrupt or undefined during process transition.

Q63. Can a system function without a dispatcher?

No. Without a dispatcher, the scheduler could select processes, but the system would have no mechanism to perform context switches or pass hardware control to user processes.

Q64. What is the difference between Multi-level Queue and Multi-level Feedback Queue Scheduling?

- **Multi-level Queue:** Divides the ready queue into fixed static queues (e.g., foreground vs. background). Processes are permanently assigned to one queue.
- **Multi-level Feedback Queue:** Allows processes to dynamically move between queues based on execution history (e.g., CPU-bound tasks drop to lower queues, while I/O-bound tasks rise).

Q65. What is Priority Inversion?

Priority Inversion is a problem where a low-priority process holds a shared resource required by a high-priority process. If a medium-priority process preempts the low-priority process, it indirectly delays the high-priority process from making progress.

Q66. What is the solution to Priority Inversion?

1. **Priority Inheritance:** The low-priority task holding the resource temporarily inherits the high priority level of the waiting task until it releases the resource.
2. **Disabling Interrupts:** Disabling interrupts while accessing the critical resource to prevent preemption.

Part 5: Process Synchronization and IPC

Q67. What is Process Synchronization?

Process Synchronization is the coordination of execution among concurrent processes to ensure correct and orderly access to shared data and resources.

Q68. What are the different ways to execute processes?

- **Concurrent Execution:** Multiple processes execute over overlapping time intervals on a single core via rapid context switching.
- **Parallel Execution:** Two or more processes execute simultaneously at the exact same instant on separate physical CPU cores.

Q69. What is a Race Condition?

A race condition occurs when multiple processes access and manipulate shared data concurrently, and the final outcome depends on the specific order in which execution steps occur.

Example: Lost update on a shared counter
`counter++` is really three steps: LOAD, ADD, STORE. If counter = 5 and two threads both LOAD 5 before either STORES, both write back 6. Two increments happened but the counter went up by only 1. A mutex or an atomic increment fixes it.

```
Thread A: LOAD  r1 <- counter   (5)
Thread B: LOAD  r2 <- counter   (5)      <- B reads before A has stored
Thread A: ADD   r1 = r1 + 1     (6)
Thread B: ADD   r2 = r2 + 1     (6)
Thread A: STORE counter <- r1   (6)
Thread B: STORE counter <- r2   (6)      <- expected 7, got 6
```

Q70. Why is Process Synchronization important?

When multiple processes write to shared memory concurrently, data corruption can occur. Synchronization enforces mutual exclusion so that only one process at a time modifies critical sections, preserving data consistency.

Q71. What are the key requirements for a Critical Section Solution?

- **Mutual Exclusion:** Only one process can execute in its critical section at any given time.
- **Progress:** If no process is in its critical section, waiting processes should not be delayed indefinitely from entering.
- **Bounded Waiting:** There must be a limit on the number of times other processes can enter their critical section after a process requests entry.

Q72. What are the main code sections of a Process in Synchronization?

- **Entry Section:** Requests permission to access the critical section.
- **Critical Section:** Code segment where shared resources are accessed and updated.
- **Exit Section:** Releases access flags and notifies waiting processes.
- **Remainder Section:** The remaining non-shared code of the process.

Q73. Can improper synchronization cause errors in an OS?

Yes, buggy synchronization logic can cause deadlocks, livelocks, race conditions, and severe performance bottlenecks.

Q74. What are Independent vs. Cooperating Processes?

- **Independent Process:** Cannot affect or be affected by other processes; does not share data (e.g., standalone calculator app).

- **Cooperating Process:** Shares data or state with other processes (e.g., a producer-consumer system like web server and browser client).

Q75. What is Inter-Process Communication (IPC)?

IPC provides mechanisms allowing different processes to communicate, exchange data, and synchronize actions across address spaces.

Q76. What are the main types of IPC mechanisms?

- **Shared Memory:** Processes read and write to a common region of physical memory.
- **Message Passing:** Processes exchange messages directly or via mailboxes/queues without sharing memory addresses.

Aspect	Shared Memory	Message Passing
Speed	Faster (no kernel call after setup)	Slower (kernel involved per message)
Synchronization	Programmer must handle it	Handled by the messaging system
Best for	Large data, same machine	Small data, distributed systems

Q77. What is a Mutex?

A Mutex (Mutual Exclusion) is a locking variable used to ensure that only one thread can acquire access to a critical resource at a time. It enforces strict ownership (only the thread that locks can unlock it).

Q78. What is a Semaphore?

A semaphore is an integer variable used as a signaling mechanism for process synchronization. It manages access to finite shared resources across multiple processes using two atomic operations: `wait()` and `signal()`.

Q79. What are the types of Semaphores?

1. **Binary Semaphore:** Value ranges strictly between 0 and 1 (similar to a Mutex, but without ownership constraints).
2. **Counting Semaphore:** Value can range over an unrestricted domain representing available units of a resource.

Q80. How is a Semaphore implemented?

Semaphores use two atomic operations:

- `wait(S)` **[P operation]:** Decrements the semaphore value. If value becomes negative, the calling process blocks.
- `signal(S)` **[V operation]:** Increments the semaphore value and wakes up a blocked process if any exist.

Q81. What are the default initial values of Semaphores?

- **Binary Semaphore:** Usually initialized to 1.
- **Counting Semaphore:** Initialized to N (total available resource units).

Q82. Is a Mutex suitable for Inter-Process Communication (IPC)?

No. Mutexes are primarily designed for thread synchronization within a single process. For multi-process IPC, shared memory with named semaphores or message queues is preferred.

> Nuance: some systems do offer process-shared or named mutexes (e.g., POSIX with PTHREAD_PROCESS_SHARED). The general interview rule still holds: mutex for threads, semaphores or message queues for processes.

Q83. What is the Critical Section in the context of Mutex?

It is the specific block of code where shared resources are updated. Mutex locking wraps around this section to ensure thread-safe serial access.

Q84. Can Mutex usage cause a Deadlock?

Yes. If thread A acquires Mutex 1 and waits for Mutex 2, while thread B acquires Mutex 2 and waits for Mutex 1, a circular deadlock occurs.

Q85. What is the difference between a Semaphore and a Mutex?

A Mutex is a locking mechanism with strict ownership (only the locking thread can unlock it). A Semaphore is a signaling mechanism where any thread can call `signal()` to unlock or increment the counter.

Aspect	Mutex	Semaphore
Type	Locking mechanism	Signaling mechanism
Value	Locked / unlocked	Integer counter
Ownership	Yes (locker must unlock)	No (any thread can signal)
Allows	One thread at a time	N threads (counting)
Typical use	Protect a critical section	Resource pools, producer-consumer ordering

Q86. What is a Spinlock?

A spinlock is a lock where a waiting process repeatedly checks (polls) in a tight loop until the lock becomes available. It avoids context-switching overhead, making it efficient for short wait times.

Q87. When should you choose Mutex over Semaphore?

Choose Mutex when managing exclusive single-thread access to a shared object and when enforcing lock ownership is necessary for code safety.

Q88. What happens if a thread forgets to release a lock?

It causes resource starvation or a deadlock, as all other threads attempting to acquire that lock will wait indefinitely.

Q89. What is an Atomic Operation?

An atomic operation executes completely as a single uninterruptible step. Intermediate states are never visible to other threads or processes.

Q90. What role do Atomic Operations play in preventing Race Conditions?

They guarantee that read-modify-write sequences (like incrementing a counter) complete without thread interruption, preventing inconsistent data states.

Q91. What is the Readers-Writers Problem?

It is a classic synchronization problem where multiple reader processes can read shared data simultaneously, but writer processes require exclusive access. The challenge is balancing concurrency while preventing writer starvation.

Q92. Can a reader access the resource while a writer is waiting?

In a writer-priority solution, new readers are blocked if a writer is waiting, preventing writer starvation. In reader-priority solutions, readers can proceed, which risks starvation for writers.

Q93. Explain the Dining Philosophers Problem.

Five philosophers sit around a table with 5 bowls of rice and 5 chopsticks. Each philosopher alternates between thinking and eating. To eat, a philosopher must acquire both left and right chopsticks. If all philosophers pick up their left chopstick simultaneously, a circular deadlock occurs.

Q94. What are the solutions to the Dining Philosophers Problem?

- Limit seating to 4 philosophers at a 5-chopstick table.
- Allow a philosopher to pick up chopsticks only if both are available simultaneously.
- Use an asymmetric strategy: even-numbered philosophers pick left chopstick first; odd-numbered pick right first.

Q95. How would you write pseudocode for the Dining Philosophers algorithm?

```
semaphore chopstick[5] = {1, 1, 1, 1, 1};
void philosopher(int i) {
  while (true) {
    think();
    wait(chopstick[i]);
    wait(chopstick[(i + 1) % 5]);
    eat();
    signal(chopstick[i]);
    signal(chopstick[(i + 1) % 5]);
  }
}
```

> This basic version can still deadlock (all five pick up their left chopstick at once). Combine it with one of the fixes from Q94, e.g., allow at most 4 philosophers to sit at the table.

Part 6: Deadlocks and Livelock

Q96. What is a Kernel-Level Thread?

A thread managed directly by the OS kernel. System calls create and schedule these threads. If one thread blocks on I/O, the kernel can schedule another thread of the same process. Example: Threads created via Windows `CreateThread()` or Linux `clone()`.

Aspect	User-Level Thread	Kernel-Level Thread
Managed by	User-space library	OS kernel
Creation/switch	Fast	Slower (system call)
Blocking I/O	Blocks the whole process	Only that thread blocks
Multi-core use	No (kernel sees one thread)	Yes

Q97. What is a Deadlock?

A deadlock occurs when two or more processes are blocked indefinitely, each waiting for a resource that is held by another process in the group.

Example: Two locks, opposite order
Thread A locks File and waits for Printer. Thread B locks Printer and waits for File. Neither can proceed. Fix: make every thread acquire locks in the same global order (File, then Printer), which breaks Circular Wait.

```
Thread A:  lock(file);     lock(printer);   ...   // holds file, wants printer
Thread B:  lock(printer);  lock(file);      ...   // holds printer, wants file  -> DEADLOCK
```

Q98. What are the four necessary conditions for Deadlock?

1. **Mutual Exclusion:** At least one resource must be held in non-shareable mode.
2. **Hold and Wait:** A process must hold at least one resource while waiting to acquire another.
3. **No Preemption:** Resources cannot be forcibly taken from a process.
4. **Circular Wait:** A closed chain of processes exists, where each process holds resources needed by the next.

All four must hold at the same time. Breaking any ONE of them prevents deadlock.

Q99. How can Deadlock be handled?

- **Deadlock Prevention:** Eliminates at least one of the 4 necessary conditions.
- **Deadlock Avoidance:** Dynamically tracks allocation states to ensure the system stays in a "safe state" (e.g., Banker's Algorithm).
- **Detection and Recovery:** Allows deadlocks to occur, detects them periodically, and resolves them via process termination or resource preemption.
- **Ignore (Ostrich Algorithm):** Used by most general-purpose OSs, since deadlocks are rare and handling them is expensive.

Q100. What is Livelock?

Livelock occurs when two or more processes continuously change their state in response to changes in the other processes without doing any useful work. Unlike deadlock, the processes are active and consume CPU cycles.

Q101. What is the Banker's Algorithm, and how does it prevent deadlock?

The Banker's Algorithm is a deadlock-avoidance algorithm that simulates resource allocation for each request. It tests whether granting the request leaves the system in a safe state where all remaining requests can be satisfied without deadlock.

Part 7: Memory Management and Virtual Memory

Q102. What are the negative effects of Livelock?

High CPU utilization with zero system throughput, application unresponsiveness, and wasted energy/resources.

Q103. What is Logical Address Space?

The set of all logical (virtual) addresses generated by the CPU for a running program.

Q104. What is Physical Address Space?

The set of all physical locations available in physical memory (RAM) corresponding to logical addresses.

Q105. What is a Logical Address?

An address reference generated by the CPU that gets mapped by the Memory Management Unit (MMU) to a physical RAM address.

Q106. What is Virtual Memory?

Virtual memory is a memory management technique that creates an illusion of a large main memory by using secondary disk storage as an extension of physical RAM.

Q107. What is Memory Allocation?

It is the process by which the operating system assigns physical or virtual memory blocks to programs for execution.

Q108. What is Contiguous Memory Allocation?

A memory allocation scheme where each process is assigned a single, continuous block of adjacent physical memory locations.

Q109. What is Non-Contiguous Memory Allocation?

A scheme where a process's memory space can be split into different chunks distributed across various physical memory locations (e.g., paging, segmentation).

Q110. What is Paging?

Paging is a non-contiguous memory management technique where physical memory is divided into fixed-size blocks called frames, and logical memory is divided into equal-sized blocks called pages. It can lead to internal fragmentation.

Q111. What is Segmentation?

Segmentation divides memory into variable-sized segments reflecting logical program modules (functions, arrays, stack). It can lead to external fragmentation.

Q112. What is the difference between Paging and Segmentation?

Paging uses fixed-size blocks determined by hardware, whereas Segmentation uses variable-size blocks based on logical program components.

Aspect	Paging	Segmentation
Block size	Fixed (page/frame)	Variable (segment)
Decided by	Hardware / OS	Programmer / compiler (logical view)
Fragmentation	Internal	External
User view	Invisible to user	Matches program structure

Q113. What is Buffering?

Buffering is storing data temporarily in a reserved memory area while it moves between devices or processes with different transfer speeds.

Q114. What is Fragmentation?

Fragmentation is the condition where memory space becomes broken into small, non-contiguous holes that reduce usable memory capacity.

Q115. What is Internal Fragmentation?

Internal fragmentation happens when memory is allocated in fixed blocks, and a process requires less space than the block size, leaving unused space inside the block.

Q116. What is External Fragmentation?

External fragmentation happens when total available memory is sufficient to satisfy a request, but the free space is scattered into non-contiguous blocks.

Q117. What is Compaction?

Compaction is a technique that shifts memory contents to combine all free memory into one large contiguous block, resolving external fragmentation.

Q118. What is a Translation Lookaside Buffer (TLB)?

The TLB is a high-speed hardware cache inside the CPU used to store recently translated page-to-frame address mappings to accelerate memory access.

Q119. What is Demand Paging?

Demand Paging is a virtual memory technique where pages are loaded into main memory only when they are referenced during execution.

Q120. What is a Page Fault?

A page fault is a hardware interrupt generated by the MMU when a program attempts to access a memory page that is not currently loaded in physical RAM.

> Page-fault handling in short: trap to OS -> check the access is valid -> find a free frame (or evict one) -> read the page from disk -> update the page table -> restart the instruction.

Q121. What is Thrashing?

Thrashing occurs when the system spends more time swapping pages in and out of disk than executing instructions, typically due to insufficient physical RAM for active working sets.

Q122. What is Swapping, and how does it differ from Paging?

Swapping moves entire processes between main memory and secondary storage. Paging operates at a fine-grained level, moving individual fixed-size pages as needed.

Q123. What are Page Replacement Algorithms?

When physical RAM is full and a page fault occurs, these algorithms decide which page frame to evict:

- **FIFO (First In First Out):** Evicts the oldest loaded page.
- **LRU (Least Recently Used):** Evicts the page that has not been accessed for the longest duration.
- **Optimal:** Evicts the page that will not be used for the longest time in the future (theoretical benchmark).

> FIFO can suffer from Belady's anomaly (more frames can cause MORE page faults). LRU and Optimal do not.

Part 8: System Calls, Kernel, Boot and Hardware

Q124. What are the common dynamic memory allocation algorithms?

- **First Fit:** Allocates the first available hole that is large enough.
- **Next Fit:** Same as First Fit, but starts searching from the last allocation point.
- **Best Fit:** Allocates the smallest hole that is big enough, minimizing leftover space.
- **Worst Fit:** Allocates the largest available hole, leaving the largest remaining hole.

Q125. What happens during BIOS loading (boot process)?

1. Power-On Self-Test (POST) runs to check hardware integrity.
2. BIOS/UEFI locates the designated bootable device.
3. Reads and runs the bootloader from the boot sector.
4. Bootloader loads the OS kernel into memory and transfers control.

Q126. What is the difference between a Kernel and an Operating System?

The kernel is the core managing component directly interfacing with hardware. It facilitates the direct bridge between hardware and software. The OS is the complete software package containing the kernel, system drivers, shell, utilities, and applications.

Q127. What are the main types of Kernels?

- **Monolithic Kernel:** All core OS services run in kernel space in a single address space (e.g., Linux).
- **Microkernel:** Keeps minimal essential services in kernel mode; drivers and file systems run in user space (e.g., L4, QNX).
- **Hybrid Kernel:** Combines microkernel modularity with monolithic speed (e.g., Windows NT, macOS XNU).

Q128. What is a System Call, and what are its main categories?

A system call provides an interface for user applications to request privileged actions from the OS kernel. Categories include: Process Control, File Management, Device Management, Information Maintenance, and Communication.

Q129. What is a Bootloader?

A bootloader is an initial startup program (e.g., GRUB, Windows Boot Manager) that loads the operating system kernel into RAM and executes it after hardware initialization finishes.

Q130. What is the difference between User Mode and Kernel Mode?

User Mode restricts direct access to hardware and memory to protect system integrity. Kernel Mode grants unrestricted execution access to CPU instructions and system hardware.

Q131. What is an Interrupt, and what are its types?

An interrupt is an event that temporarily halts normal CPU execution to process a high-priority task.

- **Hardware Interrupt:** Generated by external devices (e.g., keyboard input, disk read complete).
- **Software Interrupt:** Generated by code executing system calls or traps (e.g., divide-by-zero exception).

Q132. What is Direct Memory Access (DMA)?

DMA allows hardware devices to transfer data directly to/from main memory without routing every byte through the CPU, freeing up CPU processing capacity.

Q133. What is the trade-off between Microkernels and Monolithic Kernels?

Monolithic kernels offer high execution speed but lower fault isolation. Microkernels offer high modularity and fault tolerance but suffer from performance overhead due to frequent IPC context switches.

Part 9: File Systems, Linking and Background Processes

Q134. What is the difference between a Hard Link and a Soft Link (Symbolic Link)?

- **Hard Link:** Direct pointer to the file's underlying inode data. Remains valid even if the original filename is deleted.
- **Soft Link (Symlink):** Separate file containing a text path reference to another file. Breaks if the target file is moved or deleted.

Q135. What is a File System, and what are common examples?

A file system controls how data is stored, indexed, and retrieved on storage media. Examples include FAT32, NTFS, ext4, and APFS.

Q136. What is the difference between Static and Dynamic Linking?

Static linking combines library modules into the binary executable at compile time. Dynamic linking resolves library references at runtime using shared objects (.so / .dll), creating smaller files.

Q137. What is a Daemon process?

A daemon is a non-interactive background process that runs continuously without a direct controlling terminal to perform system tasks (e.g., cron, sshd, print spooler).

Part 10: Quick Revision Cheat Sheets

CPU scheduling algorithms at a glance

Algorithm	Preemptive?	Pros	Cons
FCFS	No	Simple, no overhead	Convoy effect, high avg waiting time
SJF	No	Minimum avg waiting time (non-preemptive)	Needs burst prediction, starvation
SRTF	Yes	Optimal avg waiting time overall	Starvation, overhead, burst prediction
Round Robin	Yes	Fair, good response time	Poor if quantum too small (overhead) or too large (acts like FCFS)
Priority	Either	Important tasks first	Starvation (fix with aging)
MLQ	Either	Separate policies per class	Rigid, starvation of lower queues
MLFQ	Yes	Adaptive, favors I/O-bound jobs	Complex to tune

Synchronization tools

Tool	Nature	Key point
Mutex	Lock with ownership	One thread, only the locker unlocks
Binary semaphore	Signal (0/1)	No ownership, any thread can signal
Counting semaphore	Signal (0..N)	Manages N identical resources
Spinlock	Busy-wait lock	Good for very short waits on multi-core
Atomic operation	Single indivisible step	Lock-free counters and flags

Memory management at a glance

Concept	Remember
Paging	Fixed-size blocks, internal fragmentation, no external fragmentation
Segmentation	Variable-size blocks, external fragmentation
Compaction	Fixes external fragmentation by moving blocks together
TLB	Cache of page-table entries, speeds up translation
Page fault	Page not in RAM, OS loads it from disk
Thrashing	More time swapping than executing; reduce multiprogramming or add RAM
Belady's anomaly	FIFO only: more frames can mean more faults

Process, thread and kernel quick facts

Topic	Quick fact
Shared among threads	Code, data, heap, open files
Private to each thread	Stack, registers, program counter
Zombie	Finished, but parent has not called wait()
Orphan	Parent died first, adopted by init/systemd

Topic	Quick fact
fork() return values	0 in child, child PID in parent, -1 on error
n sequential fork() calls	Create 2^n processes in total (2^n - 1 children)
Kernel types	Monolithic (Linux), Microkernel (QNX), Hybrid (Windows NT, macOS)

Part 11: Top 50 Operating Systems Interview Questions

The most frequently asked OS questions in campus placements and service/product company rounds. Each answer is short and structured the way you would say it aloud. Numerical and output-prediction practice follows at the end.

A. Processes and Threads (1-9)

Q1. What is the difference between a process and a thread?

A process is an independent program in execution with its own address space. A thread is a lightweight unit inside a process that shares code, data and heap with other threads but has its own stack, registers and program counter. Threads are cheaper to create and switch, but one faulty thread can crash the whole process.

Q2. What is the difference between a program and a process?

A program is a passive file of instructions on disk. A process is an active instance of it, loaded in memory with CPU state, a PCB and resources.

Q3. Explain the process states and transitions.

New -> Ready -> Running -> Terminated, with Running -> Blocked (I/O wait) -> Ready, and Running -> Ready on preemption.

Q4. What is a PCB and what does it contain?

The kernel's record for each process: PID, state, program counter, CPU registers, scheduling info, memory limits/page tables, open files and I/O status. It is what lets the OS pause and resume a process.

Q5. What is a context switch? Why is it considered overhead?

Saving the state of the running process into its PCB and loading the next process's state. It does no useful work for the user, and it also flushes caches and TLB entries, so too many switches reduce throughput.

Q6. Multiprogramming vs multitasking vs multiprocessing?

Multiprogramming: many programs in memory, switch on I/O wait. Multitasking: time slicing for responsiveness. Multiprocessing: multiple CPUs/cores running work truly in parallel.

Q7. How does fork() work? How many processes does fork(); fork(); create?

`fork()` duplicates the caller; the child gets a copy of the parent's address space (using copy-on-write). Return value: 0 in child, child PID in parent. Two sequential forks produce 2^2 = 4 processes in total.

Q8. Zombie vs orphan process?

Zombie: child has finished but the parent has not yet called `wait()`, so its process-table entry remains. Orphan: parent exited first, and init/systemd adopts the child.

Q9. User-level vs kernel-level threads?

User-level: managed by a library, fast, but one blocking call blocks the whole process and it cannot use multiple cores. Kernel-level: managed by the OS, can run in parallel on multiple cores and block independently, but cost more to create and switch.

B. CPU Scheduling (10-18)

Q10. Scheduler vs dispatcher?

The scheduler decides which process runs next. The dispatcher performs the actual hand-over: context switch, switch to user mode, jump to the right instruction. The delay is dispatch latency.

Q11. Preemptive vs non-preemptive scheduling?

Preemptive: OS can take the CPU away (RR, SRTF, preemptive priority). Non-preemptive: process keeps the CPU until it finishes or blocks (FCFS, SJF).

Q12. Long-term, short-term and medium-term schedulers?

Long-term controls admission into memory (degree of multiprogramming). Short-term picks the next ready process (runs most often). Medium-term swaps processes out and in.

Q13. Which scheduling algorithm gives the minimum average waiting time?

SJF for non-preemptive, and SRTF overall (provably optimal). In practice they are hard to implement because burst times must be predicted.

Q14. What is the convoy effect?

In FCFS, a long CPU-bound process at the front makes all short processes wait behind it, increasing average waiting time and lowering utilization of I/O devices.

Q15. What is starvation and how does aging solve it?

Starvation: a low-priority or long process never gets the CPU. Aging gradually raises the priority of waiting processes so they eventually run.

Q16. How do you choose the time quantum in Round Robin?

Too small: excessive context switching. Too large: degenerates into FCFS and responsiveness suffers. A common rule is that about 80% of CPU bursts should be shorter than the quantum.

Q17. Multi-level queue vs multi-level feedback queue?

MLQ: fixed queues, processes never move (e.g., foreground and background). MLFQ: processes move between queues based on behavior, so CPU hogs sink and interactive/I/O-bound jobs stay at high priority. Modern general-purpose OSs use MLFQ-like schemes.

Q18. What is priority inversion and how is it solved?

A high-priority task waits for a lock held by a low-priority task, which is itself preempted by a medium-priority task. Solved with priority inheritance (or priority ceiling). The Mars Pathfinder bug in 1997 was a famous case.

C. Synchronization and Deadlocks (19-30)

Q19. What is a race condition? How do you prevent it?

Outcome depends on the interleaving of concurrent accesses to shared data. Prevent with mutual exclusion (mutex/lock), semaphores, monitors or atomic operations.

Q20. What are the three requirements of a critical-section solution?

Mutual exclusion, progress, and bounded waiting.

Q21. Mutex vs semaphore?

Mutex: a lock with ownership, for mutual exclusion on one resource. Semaphore: a counter-based signaling mechanism with no ownership, for controlling access to N resources or for ordering events between threads.

Q22. Binary semaphore vs mutex?

Both take two states, but a mutex must be unlocked by the thread that locked it. A binary semaphore can be signaled by a different thread, so it suits signaling, not just locking.

Q23. Spinlock vs mutex - when to use which?

Spinlock busy-waits and avoids a context switch, so it wins when the critical section is very short and you have multiple cores. A mutex puts the waiter to sleep, so it is better when the wait may be long.

Q24. Solve the Producer-Consumer (bounded buffer) problem with semaphores.

Use `empty` (free slots), `full` (filled slots) and `mutex` (buffer access). Order of `wait` calls matters: reversing them can deadlock.

```
semaphore mutex = 1, empty = N, full = 0;

producer() {                          consumer() {
  while (true) {                        while (true) {
    item = produce();                     wait(full);          // wait for an item
    wait(empty);                          wait(mutex);
    wait(mutex);                          item = remove();
    insert(item);                         signal(mutex);
    signal(mutex);                        signal(empty);       // free a slot
    signal(full);                         consume(item);
  }                                     }
}                                     }
```

Q25. What is the readers-writers problem?

Many readers may read together, but a writer needs exclusive access. Reader-priority can starve writers, writer-priority can starve readers; fair variants use queues or timestamps.

Q26. Explain the Dining Philosophers problem and one solution.

Five philosophers need two chopsticks each; if all grab the left one, they deadlock. Solutions: allow only 4 at the table, pick up both chopsticks atomically, or make odd and even philosophers pick in opposite order.

Q27. What are the four conditions for deadlock?

Mutual exclusion, hold and wait, no preemption, circular wait. All four must hold together.

Q28. How can deadlocks be prevented, avoided and detected?

Prevention: break one condition (e.g., order resources to break circular wait, request all resources at once to break hold-and-wait). Avoidance: Banker's algorithm keeps the system in a safe state. Detection: build a wait-for graph or run a detection algorithm, then recover by killing a process or preempting resources.

Q29. Explain the Banker's algorithm.

Each process declares its maximum need. For every request the OS pretends to grant it and checks whether a safe sequence exists, meaning all processes can finish in some order. If not, the request waits. Key formula: Need = Max - Allocation.

Q30. Deadlock vs livelock vs starvation?

Deadlock: processes blocked forever, no CPU use. Livelock: processes keep running and reacting to each other but make no progress. Starvation: one process is repeatedly denied a resource while others proceed.

D. Memory Management (31-40)

Q31. What is virtual memory and why is it useful?

It gives every process a large, private, contiguous address space by mapping virtual pages to physical frames and using disk as overflow. Benefits: run programs bigger than RAM, process isolation, and efficient sharing.

Q32. Paging vs segmentation?

Paging: fixed-size pages, invisible to programmer, internal fragmentation. Segmentation: variable-size logical units (code, stack, heap), external fragmentation. Many systems combine both.

Q33. Internal vs external fragmentation and their fixes?

Internal: wasted space inside an allocated block (fixed partitions, paging); reduce with smaller pages. External: free memory split into small holes; fix with compaction, paging or segmentation with paging.

Q34. What happens on a page fault?

MMU traps to the OS -> OS verifies the reference is legal -> finds a free frame or evicts a victim page -> schedules a disk read -> updates the page table -> restarts the faulting instruction.

Q35. What is thrashing? How do you prevent it?

Constant paging with little useful work because processes lack enough frames for their working sets. Fix by using the working-set model, page-fault-frequency control, reducing the degree of multiprogramming, or adding RAM.

Q36. Compare FIFO, LRU and Optimal page replacement. What is Belady's anomaly?

FIFO evicts the oldest page (simple, but may evict hot pages). LRU evicts the least recently used (good approximation of optimal, costlier to implement). Optimal evicts the page used farthest in the future (benchmark only). Belady's anomaly: with FIFO, adding frames can increase page faults.

Q37. What is a TLB and how do you compute effective access time?

TLB caches recent page-to-frame translations. With hit ratio h, TLB time t and memory time m: EAT = h x (t + m) + (1 - h) x (t + 2m). Example: t = 10 ns, m = 100 ns, h = 0.9 gives 0.9 x 110 + 0.1 x 210 = 120 ns.

Q38. What is demand paging?

Pages are loaded only when first referenced (lazily), using a valid/invalid bit in the page table. It saves memory and start-up time at the cost of page faults.

Q39. Swapping vs paging?

Swapping moves whole processes between RAM and disk. Paging moves individual pages, so only the needed part of a process is in memory.

Q40. First fit vs best fit vs worst fit?

First fit: first hole that fits (fast). Best fit: smallest hole that fits (leaves tiny unusable holes). Worst fit: largest hole (leaves large remainders). First fit is usually fastest and as good as best fit in practice.

E. Kernel, System Calls, Files and Misc (41-50)

Q41. Kernel vs operating system?

The kernel is the core program that manages CPU, memory, devices and system calls. The OS is the kernel plus the shell, libraries, drivers, utilities and applications around it.

Q42. Monolithic vs microkernel?

Monolithic: all services in kernel space, fast but one buggy driver can crash everything (Linux). Microkernel: minimal kernel, services in user space, safer and modular but slower due to IPC (QNX, L4). Hybrid kernels mix both (Windows NT, macOS).

Q43. User mode vs kernel mode?

User mode cannot execute privileged instructions or touch hardware directly. Kernel mode can. The CPU mode bit protects the system from faulty or malicious programs.

Q44. What is a system call? Walk through one.

The controlled gateway from user programs to the kernel. Example `read()`: the library places the syscall number and arguments in registers -> executes a trap instruction -> CPU switches to kernel mode -> kernel runs the handler -> result is returned -> CPU returns to user mode.

Q45. Interrupt vs trap (exception) vs system call?

Interrupt: asynchronous, from hardware (keyboard, disk). Trap/exception: synchronous, caused by the running instruction (divide by zero, page fault). System call: a deliberate software trap requested by the program.

Q46. What is DMA and why is it needed?

A DMA controller moves blocks of data between a device and memory without the CPU copying each byte; the CPU is interrupted only when the whole transfer completes. This frees the CPU for other work.

Q47. Explain the boot process.

Power on -> BIOS/UEFI runs POST -> finds the boot device -> loads the bootloader (GRUB, Windows Boot Manager) -> bootloader loads the kernel into RAM -> kernel initializes devices and starts the first process (init/systemd).

Q48. Hard link vs soft link?

Hard link: another directory name for the same inode; survives deletion of the original name, cannot cross file systems. Soft link: a separate file holding a path; becomes dangling if the target is removed; can cross file systems and point to directories.

Q49. Static vs dynamic linking?

Static: libraries copied into the executable at build time (self-contained, larger). Dynamic: libraries (.so/.dll) loaded at run time and shared by programs (smaller, easier to update, but dependency on installed versions).

Q50. What is the difference between stack and heap in a process? What is a daemon?

Stack: automatic, per-thread memory for function calls and locals (fast, LIFO, limited size). Heap: dynamic memory shared by all threads (`malloc`/`new`), managed manually or by a garbage collector. A daemon is a background process with no controlling terminal (cron, sshd).

Bonus: Solved numerical and output questions

B1. Scheduling: compare FCFS, SJF, SRTF and Round Robin (q = 2)

Processes: P1 (arrival 0, burst 8), P2 (arrival 1, burst 4), P3 (arrival 2, burst 2). Waiting time = completion - arrival - burst.

Algorithm	Execution order (Gantt)	Waiting times P1, P2, P3	Average WT
FCFS	P1 0-8, P2 8-12, P3 12-14	0, 7, 10	5.67
SJF (non-preemptive)	P1 0-8, P3 8-10, P2 10-14	0, 9, 6	5.00
SRTF (preemptive)	P1 0-1, P2 1-2, P3 2-4, P2 4-7, P1 7-14	6, 2, 0	2.67
Round Robin (q = 2)	P1 0-2, P2 2-4, P3 4-6, P1 6-8, P2 8-10, P1 10-14	6, 5, 2	4.33

SRTF gives the best average, as expected. Round Robin trades a slightly higher average for fairness and response time.

B2. Banker's algorithm: is the system safe?

One resource type with 12 units. P1: max 10, allocated 5. P2: max 4, allocated 2. P3: max 9, allocated 2. Available = 12 - 9 = 3. Need = Max - Allocation: P1 = 5, P2 = 2, P3 = 7. Step 1: P2 needs 2 <= 3, so it runs and releases its 4 units; available becomes 5. Step 2: P1 needs 5 <= 5, so it runs and releases 10; available becomes 10. Step 3: P3 needs 7 <= 10, so it runs. A safe sequence exists: **P2, P1, P3**, so the state is safe.

B3. How many times is "Hi" printed?

```
int main() {
    fork();
    fork();
    printf("Hi\n");        // executed by every process
    return 0;
}
// Prints "Hi" 4 times: after 2 forks there are 2^2 = 4 processes
```

B4. What is the output order?

```
if (fork() == 0) {
    printf("Child\n");
} else {
    wait(NULL);             // parent waits for the child to finish
    printf("Parent\n");
}
// Output: Child, then Parent (wait() guarantees the order, and reaps the child so it cannot become a
    // zombie)
```

Final advice: for every concept give (1) a one-line definition, (2) the key difference or trade-off, and (3) one small example when it helps, such as the two-lock deadlock, the counter race condition, or the convoy numbers. Practice the scheduling and Banker's numericals by hand - they are asked in almost every written round.